Правительство Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»
(НИУ ВШЭ)

Московский институт электроники и математики им. А.Н. Тихонова

ОТЧЕТ
О ПРАКТИЧЕСКОЙ РАБОТЕ № 1
по дисциплине «Криптографические методы защиты информации»
Современные симметричные шифры

Студент гр. МКБ251 ВШЭ
_____ В.В. Созыкин
«___» _____ 2026 г.

Руководитель
Заведующий кафедрой информационной безопасности киберфизических систем
канд. техн. наук, доцент
_____О.О. Евсютин
«___» _____ 2026 г.

Москва 2026

СОДЕРЖАНИЕ

1 Задание на практическую работу 3

2 Теоретическая часть 4

2.1 Стандарт ГОСТ Р 34.12-2015 4

3 Программная реализация программы «Магма» 5

3.1 Описание разработки программы «Магма» 5

3.2 Листинг кода программы «Магма» 5

3.3 Запуск 8

3.4 Результаты работы 9

4 Выводы о проделанной работе 13

5 Генератор ключей (genkey.py) 13

5.1 Назначение генератора ключей 13

5.2 Используемые библиотеки 13

5.3 Программная реализация генератора ключей 13

5.4 Описание разработки 13

5.5 Листинг кода генератора ключей 14

5.6 Запуск 15

5.7 Результаты работы 15

5.8 Выводы о проделанной работе 18

6 Список использованных источников: 19

1 Задание на практическую работу

Целью данной работы является приобретение навыков программной реализации современных алгоритмов симметричного шифрования.

В рамках практической работы необходимо выполнить следующее:

Написать программную реализацию одного из перечисленных ниже симметричных шифров (по выбору студента). Реализация шифра должны быть выполнена студентом самостоятельно без использования готовых библиотечных функций, напрямую реализующих алгоритм шифрования или его отдельные этапы. Варианты шифров:

- Магма;
- Кузнечик;
- AES;

Подготовить отчет о выполнении работы.

2 Теоретическая часть

2.1 Стандарт ГОСТ Р 34.12-2015

В Российской Федерации основным документом, регламентирующим алгоритмы блочного шифрования, является ГОСТ Р 34.12-2015 «Информационная технология. Криптографическая защита информации. Блочные шифры». Данный стандарт пришел на смену устаревшему ГОСТ 28147-89 и унифицировал требования к современным криптографическим средствам. Стандарт утверждает два базовых алгоритма:

1) «Кузнечик» (Kuznyechik): длина блока 128 бит, используется SP-сеть;
2) «Магма» (Magma): длина блока 64 бита, используется структура сети Фейстеля.

Особенностью нового стандарта является отказ от описания режимов работы (они вынесены в отдельный ГОСТ Р 34.13-2015) и фиксация узлов замены (S-боксов) для алгоритма «Магма», которые ранее были секретным или сменным параметром.

2.2 Описание алгоритма «Магма»

Шифр «Магма» является итерационным блочным шифром с длиной блока n=64 бита и длиной ключа k=256 бит. Алгоритм строится на основе схемы Фейстеля и включает 32 раунда преобразований. В каждом раунде блок данных делится на две части (L и R). Правая часть преобразуется функцией g, складывается с левой частью по модулю 2 (XOR), после чего части меняются местами. Ключевые элементы алгоритма:

1) сложение с раундовым ключом по модулю 232,
2) нелинейная замена 4-битных подблоков по фиксированной таблице π (S-box),
3) циклический сдвиг результата на 11 бит влево.

3 Программная реализация программы «Магма»

3.1 Описание разработки программы «Магма»

В ходе выполнения работы было разработано консольное приложение на языке Python. Выбор языка обусловлен удобством работы с байтовыми строками и наличием встроенных функций для манипуляции битами (сдвиги, логические операции), что необходимо для реализации криптографических примитивов. Сторонние библиотеки шифрования (pycryptodome, cryptography) не использовались, алгоритм реализован «с нуля».

Архитектура приложения включает следующие функции:

1) str_to_key — парсинг 256-битного ключа из шестнадцатеричной строки;
2) g_function — реализация раундовой функции g (сложение с ключом, S-блок, сдвиг);
3) magma_cycle — основной цикл из 32 раундов сети Фейстеля;
4) process_file — функция чтения файла в бинарном режиме, дополнения последнего блока (padding) нулями до 64 бит и записи результата.

3.2 Листинг кода программы «Магма»

https://github.com/VasiliySozykin/Practical-work-on-the-topic-Symmetric-ciphers-by-V.V.-Sozykin/blob/main/mycipher.py

```
import sys
import os
import struct

# (S-Box) согласно ГОСТ Р 34.12-2015
Pi = [
    [12, 4, 6, 2, 10, 5, 11, 9, 14, 8, 13, 7, 0, 3, 15, 1],
    [6, 8, 2, 3, 9, 10, 5, 12, 1, 11, 7, 13, 0, 4, 15, 14],
    [7, 11, 5, 8, 12, 4, 2, 0, 14, 1, 3, 10, 9, 15, 6, 13],
    [13, 1, 7, 4, 11, 5, 0, 15, 3, 12, 14, 6, 9, 10, 2, 8],
    [5, 10, 15, 12, 1, 13, 14, 11, 8, 3, 6, 0, 4, 7, 9, 2],
    [14, 5, 0, 15, 13, 11, 3, 6, 9, 2, 12, 7, 1, 8, 10, 4],
    [11, 13, 12, 3, 7, 14, 10, 5, 0, 9, 4, 15, 2, 8, 1, 6],
    [15, 12, 9, 7, 3, 0, 11, 4, 1, 1, 2, 13, 6, 10, 8, 5]

]
```

```
def str_to_key(key_str):
    #Преобразует hex-строку ключа в список из 8-ми 32-битных чисел.
    key_bytes = bytes.fromhex(k4ey_str)
    if len(key_bytes) != 32:
        raise ValueError("Ошибка: Длина ключа должна быть ровно 32 байта (64 hex-
символа).")
    # Разбиваем 256 бит на 8 подключей по 32 бита
    return [int.from_bytes(key_bytes[i:i+4], 'big') for i in range(0, 32, 4)]

def get_round_keys(master_key):
    #Развертывание ключей: 32 раунда.

    keys = []
    # 3 цикла прямого порядка (1-24 раунды)
    for _ in range(3):
        keys.extend(master_key)
    # 1 цикл обратного порядка (25-32 раунды)
    keys.extend(master_key[::-1])
    return keys
def t_transformation(data):
    #Нелинейное биективное преобразование (S-блоки)
    output = 0
    # Проходим по 8 полубайтам (4 бита) числа
    for i in range(8):
        # Извлекаем 4 бита
        part = (data >> (4 * i)) & 0xF
        # Заменяем по таблице и ставим на место
        output |= (Pi[i][part] << (4 * i))
    return output
def g_function(data, key):
    internal = (data + key) % (2 ** 32)
    substituted = t_transformation(internal)
    # Циклический сдвиг влево на 11
    result = ((substituted << 11) | (substituted >> 21)) & 0xFFFFFFFF
    return result
```

```python
def magma_cycle(block_bytes, round_keys, mode='encrypt'):
    # Разбиваем блок на левую (L) и правую (R) части
    L = int.from_bytes(block_bytes[:4], 'big')
    R = int.from_bytes(block_bytes[4:], 'big')

    if mode == 'decrypt':
        # Для расшифровки ключи те же, но идут в обратном порядке
        round_keys = round_keys[::-1]
    for i in range(31):
        temp = R
        R = L ^ g_function(R, round_keys[i])
        L = temp
    temp = R
    R = L ^ g_function(R, round_keys[31])
    L = temp
    # Для корректности с вектором тестов возвращаем склеенный блок
    return R.to_bytes(4, 'big') + L.to_bytes(4, 'big')
def process_file(filepath, key_hex, mode='encrypt'):
    out_path = filepath + ('.enc' if mode == 'encrypt' else '.dec')
    keys_list = str_to_key(key_hex)
    round_keys = get_round_keys(keys_list)
    print(f"Начинаю {mode} файла: {filepath}")
    with open(filepath, 'rb') as f_in, open(out_path, 'wb') as f_out:
        while True:
            chunk = f_in.read(8)
            if not chunk:
                break
            # Дополнение (Padding) если блок меньше 8 байт
            original_len = len(chunk)
            if len(chunk) < 8:
                chunk += b'\x00' * (8 - len(chunk))
            # Обработка блока
            processed_block = magma_cycle(chunk, round_keys, mode)
            f_out.write(processed_block)
    print(f"Готово! Результат записан в: {out_path}")
```

```
def main():
    while True:
        choice = input("\nВыберите действие (1-шифровать, 2-дешифровать, q-выход): ")
        if choice == 'q':
            break
        if choice not in ['1', '2']:
            continue
        mode = 'encrypt' if choice == '1' else 'decrypt'
        fpath = input("Введите путь к файлу: ").strip('"')
        if not os.path.exists(fpath):
            print("Файл не найден!")
            continue
        print("Введите ключ (64 символа hex).")
        print("Пример: ffeeddccbbaa9988776655443322110 0f0f1f2f3f4f5f6f7f8f9fafbfcfdfeff")
        key_input = input("Ключ: ").strip()
try:
            process_file(fpath, key_input, mode)
        except Exception as e:
            print(f"Ошибка: {e}")
if __name__ == '__main__':
    main()
```

3.3 Запуск

1) Сохранить код в файл, например mycipher.py.
2) Запустить в терминале: python3 mycipher.py.

Дополнительные инструкции:

а) выбрать шифровать,

б) указать путь к файлу (myfile.txt),

в) предоставить ключ, например: abcdef1234567890abcdef1234567890abcdef1234567890abcdef1234567890,

г) выбрать дешифровать,

д) указать путь к файлу (myfile.txt.enc);

e) предоставить ключ, например: abcdef1234567890abcdef1234567890abcdef1234567890abcdef1234567890.

3.4 Результаты работы

1) Программа была протестирована на текстовом файле.

2) Был создан файл myfile.txt с открытым текстом.

3) Произведено шифрование с произвольным ключом.

4) Получен файл myfile.txt.enc (нечитаемый бинарный набор данных).

5) Произведена расшифровка обратно в myfile.txt.dec. Содержимое совпало с оригиналом.

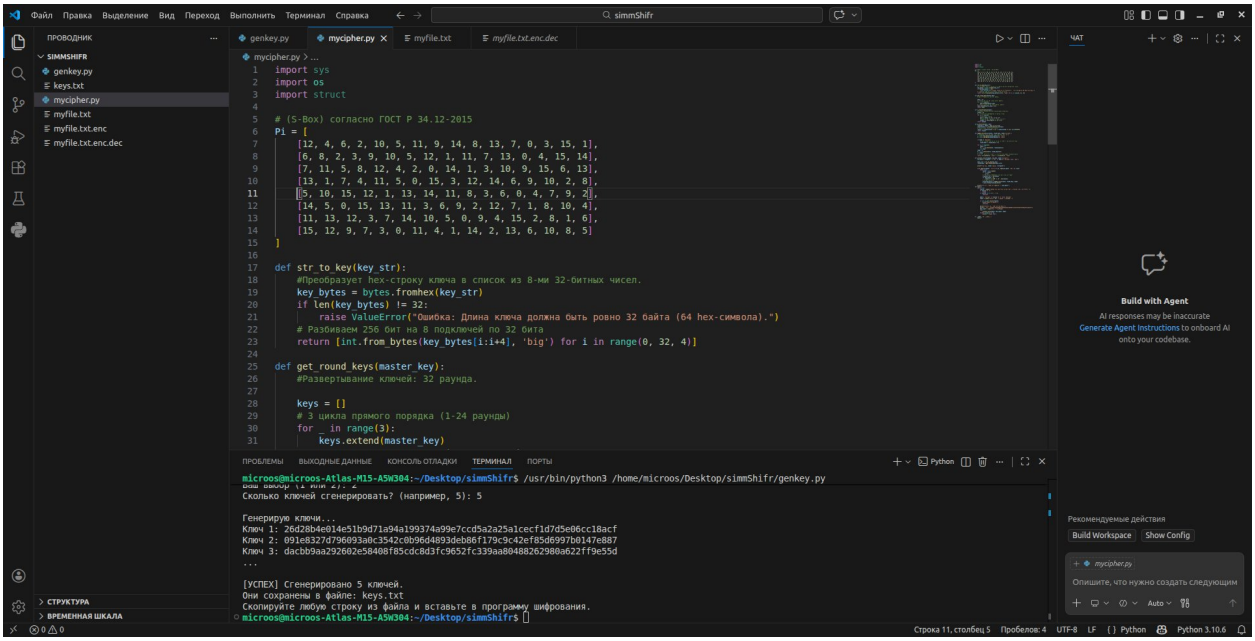

Рисунок 1 – Рабочее окно VScode

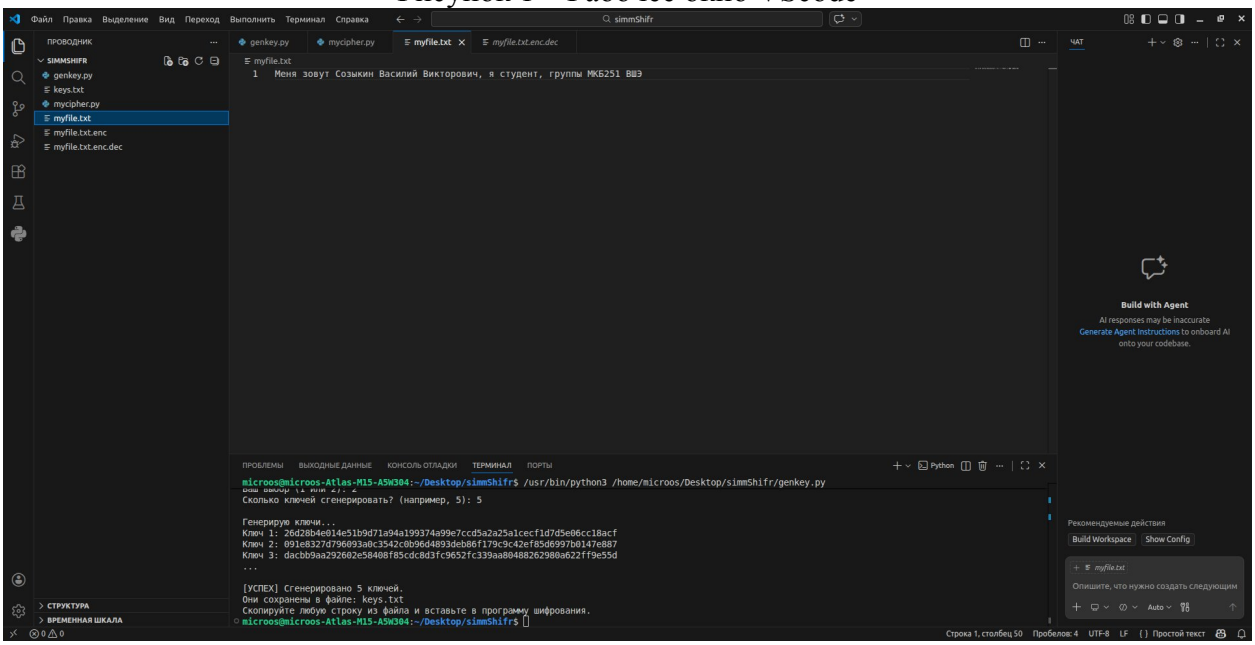

Рисунок 2 – создание текстового файла в VScode

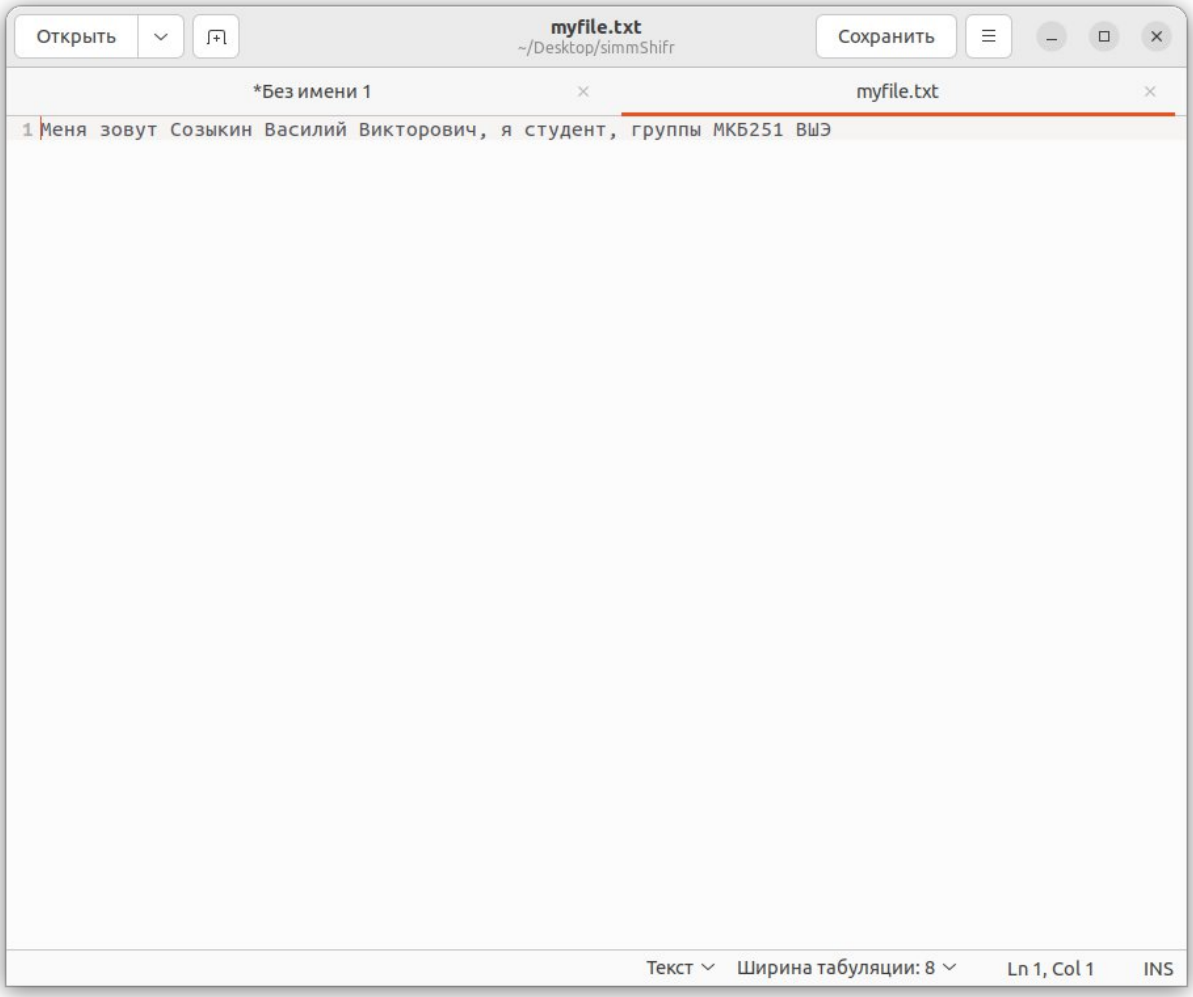

Рисунок 3 – Содержимое файла myfile.txt до шифрования

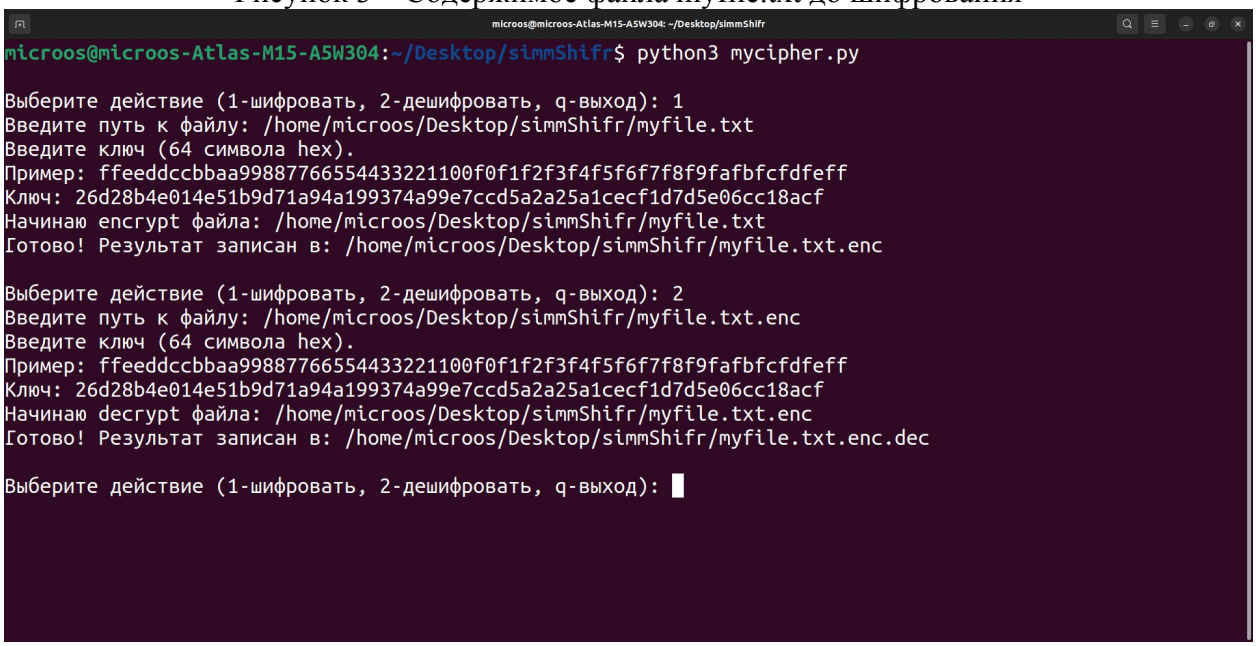

Рисунок 4 – Пример работы программы , шифрование, дешифрование

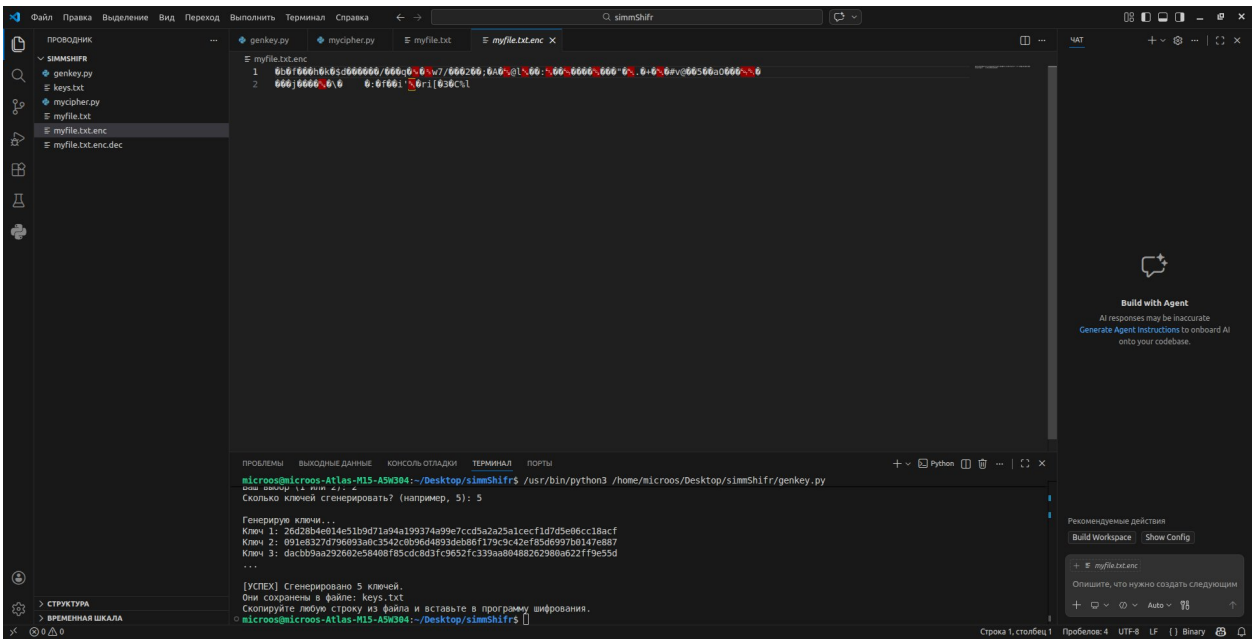

Рисунок 5 – Содержимое файла myfile.txt.enc после шифрования в VScode

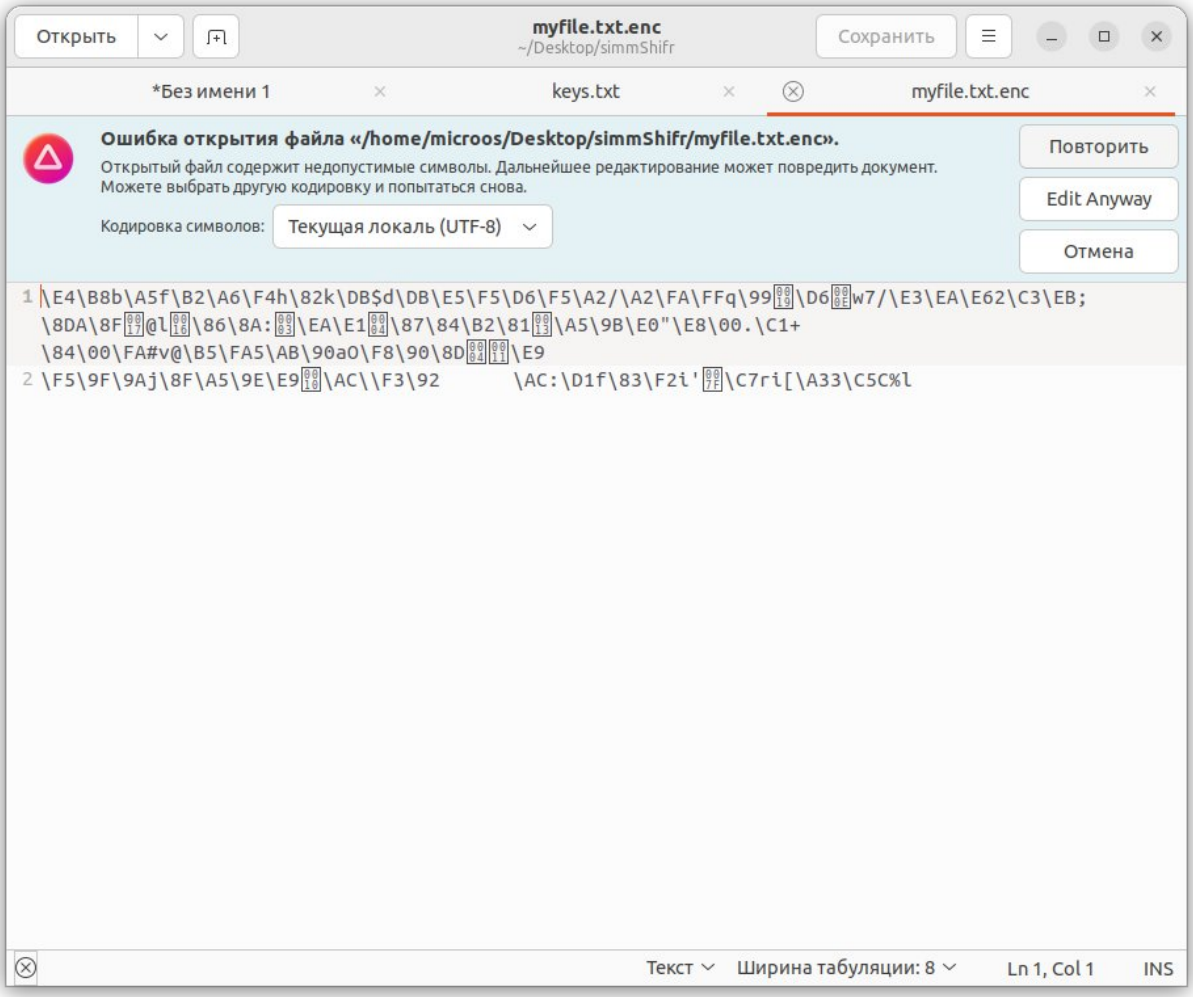

Рисунок 6 – Содержимое файла myfile.txt.enc после шифрования в текстовом файле

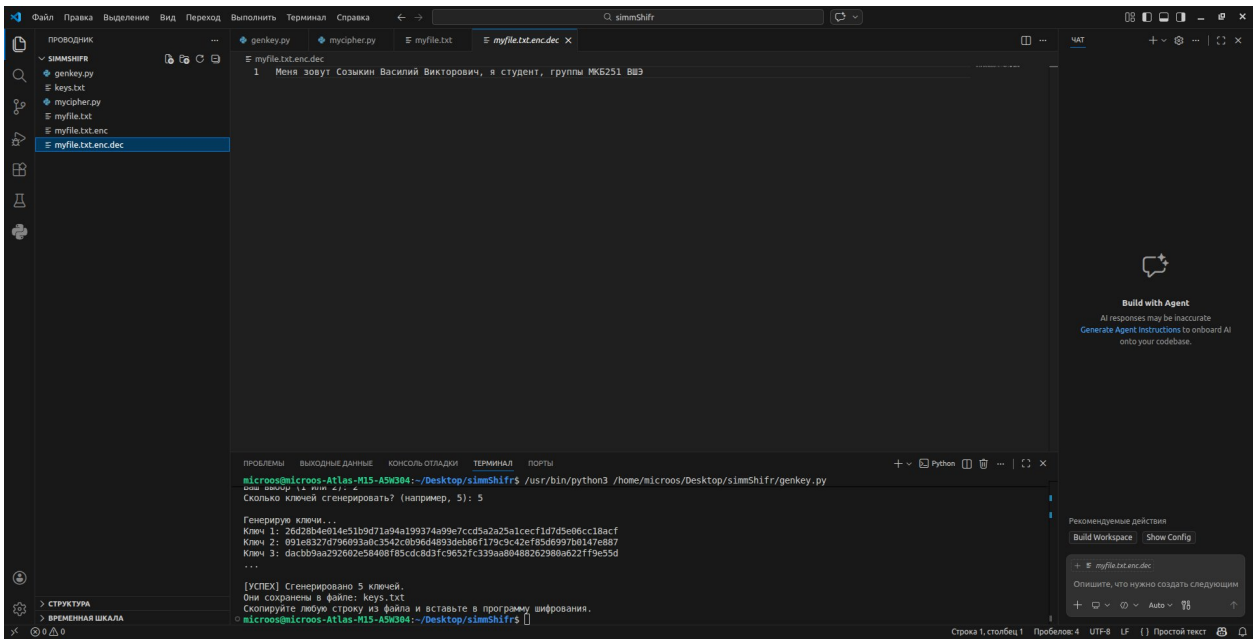

Рисунок 7 – Содержимое файла myfile.txt.enc.dec после дешифрования в VScode

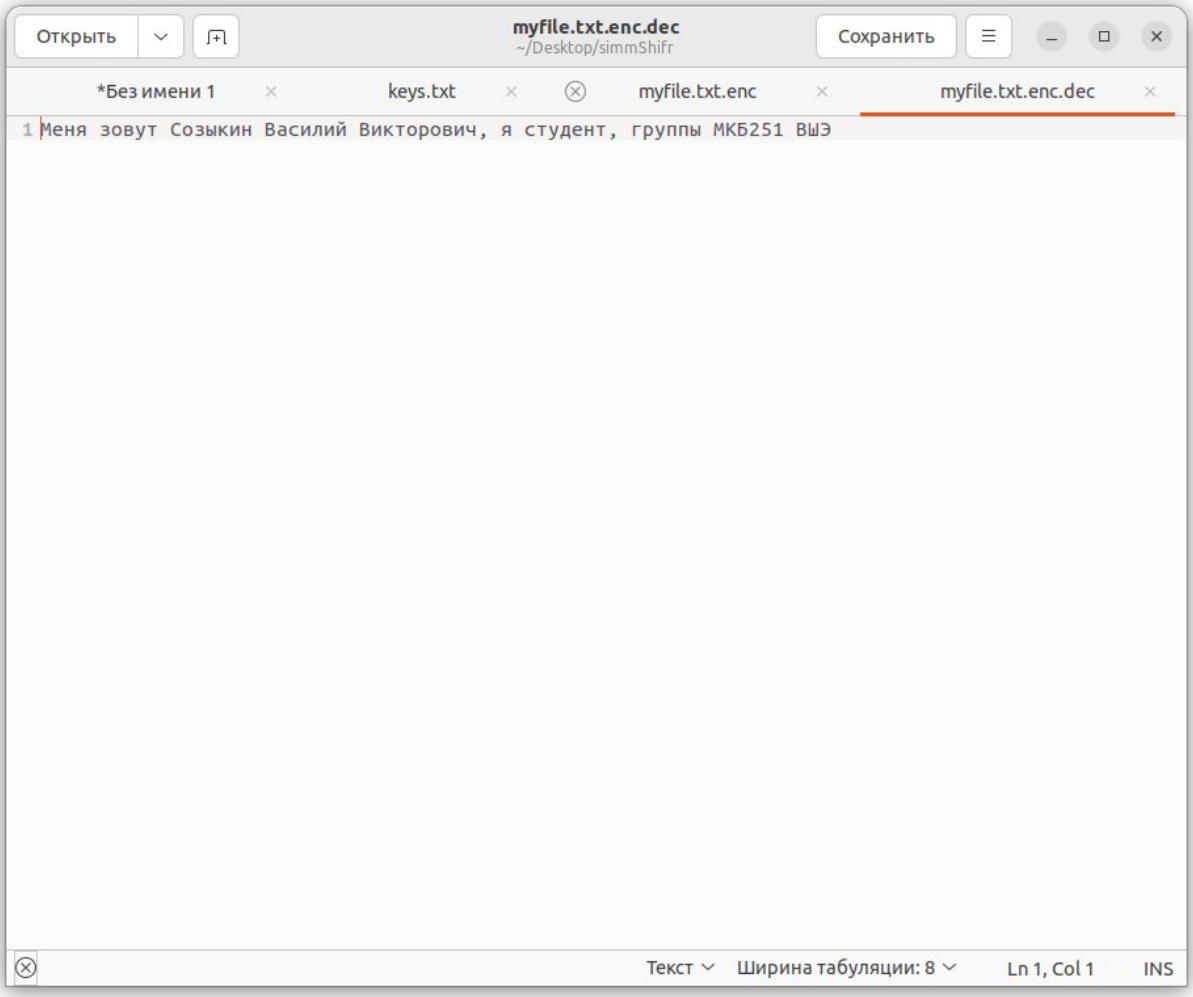

Рисунок 7 – Содержимое файла myfile.txt.enc.dec после дешифрования в текстовом файле

4 Выводы о проделанной работе

В ходе лабораторной работы были изучены принципы построения современных блочных шифров. На практике реализован алгоритм «Магма» (ГОСТ Р 34.12-2015). Получены навыки работы с битовыми операциями (XOR, сдвиги), табличными заменами и реализацией сети Фейстеля. Программная реализация корректно шифрует и расшифровывает файлы любых форматов.

5 Генератор ключей (genkey.py)

5.1 Назначение генератора ключей

Скрипт предназначен для автоматической генерации криптографически стойких ключей шифрования. Основная цель — создать случайную последовательность байтов и представить её в шестнадцатеричном формате (HEX), который требуется для работы алгоритмов AES и «Кузнечик».

Использование генератора исключает человеческие ошибки (например, ввод недопустимых символов g, z, k в HEX-строку) и гарантирует высокую энтропию ключа.

5.2 Используемые библиотеки

Secrets: Стандартная библиотека Python (начиная с версии 3.6), предназначенная для генерации криптографически сильных случайных чисел. В отличие от модуля random, который использует псевдослучайные числа (PRNG) и предсказуем, secrets использует источники энтропии операционной системы (например, /dev/urandom в Linux), что делает ключи безопасными для реального шифрования.

5.3 Программная реализация генератора ключей

5.4 Описание разработки

1) Выбор длины ключа: Пользователю предлагается выбрать один из стандартов:

а) 128 бит (16 байт): Генерируется строка из 32 шестнадцатеричных символов. Используется для AES-128;

б) 56 бит (32 байта): Генерируется строка из 64 шестнадцатеричных символов. Используется для AES-256 и ГОСТ 34.12-2015 («Кузнечик»).

2) Генерация: С помощью функции secrets.token_hex(nbytes) создается случайная последовательность байтов заданной длины и автоматически конвертируется в строку HEX.

3) Сохранение: Сгенерированные ключи выводятся в консоль для быстрого копирования и дублируются в файл keys.txt в текущей директории.

5.5 Листинг кода генератора ключей

https://github.com/VasiliySozykin/Practical-work-on-the-topic-Symmetric-ciphers-by-V.V.-Sozykin/blob/main/genkey.py

```
import secrets
def generate_keys():
    print("=== Генератор криптографических ключей  ===")
    # 1. Выбор длины ключа
    print("\nВыберите длину ключа:")
    print("1 - 128 бит (32 символа hex) -> Для AES-128")
    print("2 - 256 бит (64 символа hex) -> Для AES-256 или Кузнечика")
    choice = input("Ваш выбор (1 или 2): ").strip()
    if choice == '1':
        num_bytes = 16  # 128 бит
        algo_name = "AES-128"
    elif choice == '2':
        num_bytes = 32  # 256 бит
        algo_name = "AES-256 / Kuznechik"
    else:
        print("Ошибка: Неверный выбор. Использую по умолчанию 256 бит.")
        num_bytes = 32
        algo_name = "256-bit"
    # 2. Количество ключей
    try:
        count = int(input("Сколько ключей сгенерировать? (например, 5): "))
    except ValueError:
        count = 1
    filename = "keys.txt"
    # 3. Генерация и запись
    with open(filename, "w", encoding="utf-8") as f:
        f.write(f"--- Generated Keys for {algo_name} ---\n")
        print("\nГенерирую ключи...")
        for i in range(count):
            # secrets.token_hex(n) возвращает строку из 2*n hex-символов
            key = secrets.token_hex(num_bytes)
            # Записываем в файл
```

```
        f.write(f"{key}\n")
        # Выводим на экран (первые пару штук)
        if i < 3:
            print(f"Ключ {i+1}: {key}")
    if count > 3:

        print("...")
  print(f"\n[УСПЕХ] Сгенерировано {count} ключей.")
  print(f"Они сохранены в файле: {filename}")
  print("Скопируйте любую строку из файла и вставьте в программу шифрования.")
if __name__ == "__main__":
  generate_keys()
```

5.6 Запуск

1) Сохранить код в файл, keygen.py,

2) Запустите скрипт командой: bash python3 keygen.py,

а) в меню выберите тип шифрования (введите 1 для AES-128 или 2 для Кузнечика/AES-256),

б) введите количество необходимых ключей (например, 5).

Ключи можно взять в консоли либо открыть созданный файл keys.txt, скопировать любую строку и использовать её в программе шифрования.

5.7 Результаты работы

1) Программа была протестирована в консоли Linux, был создан файл keys.txt с типом шифрования (для Кузнечика/AES-256).

2) Произведено шифрование с полученным ключом, ключ отработал корректно.

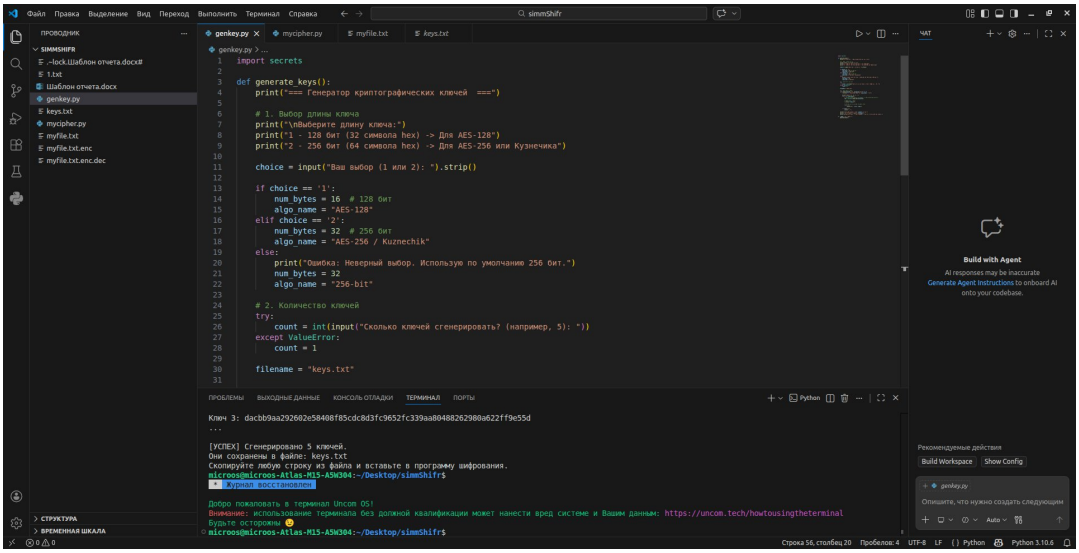

Рисунок 8 – Рабочее окно VScode

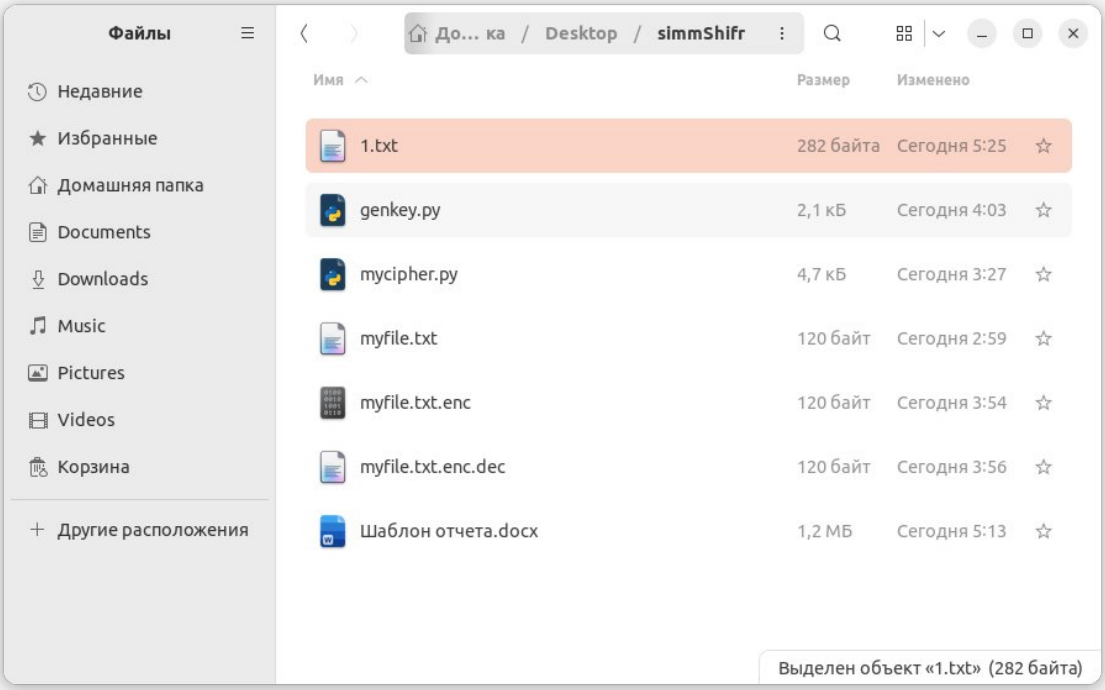

Рисунок 9 – Сохраненный скрирпт genkey.py

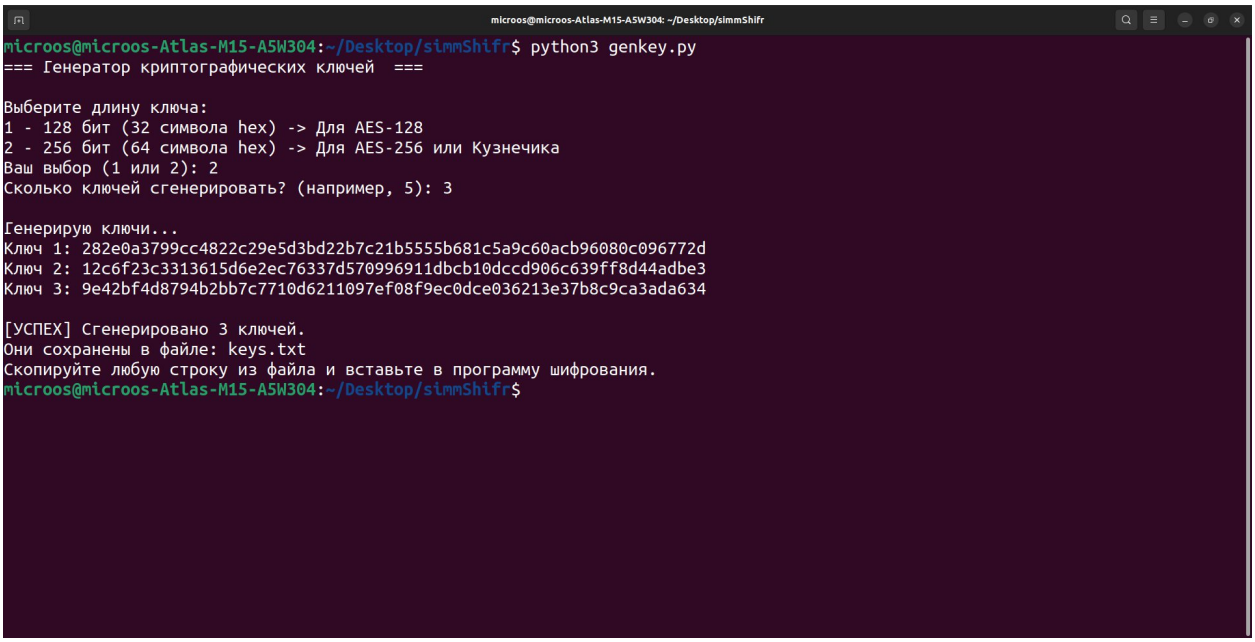

Рисунок 10 – запуск скрирпта genkey.py

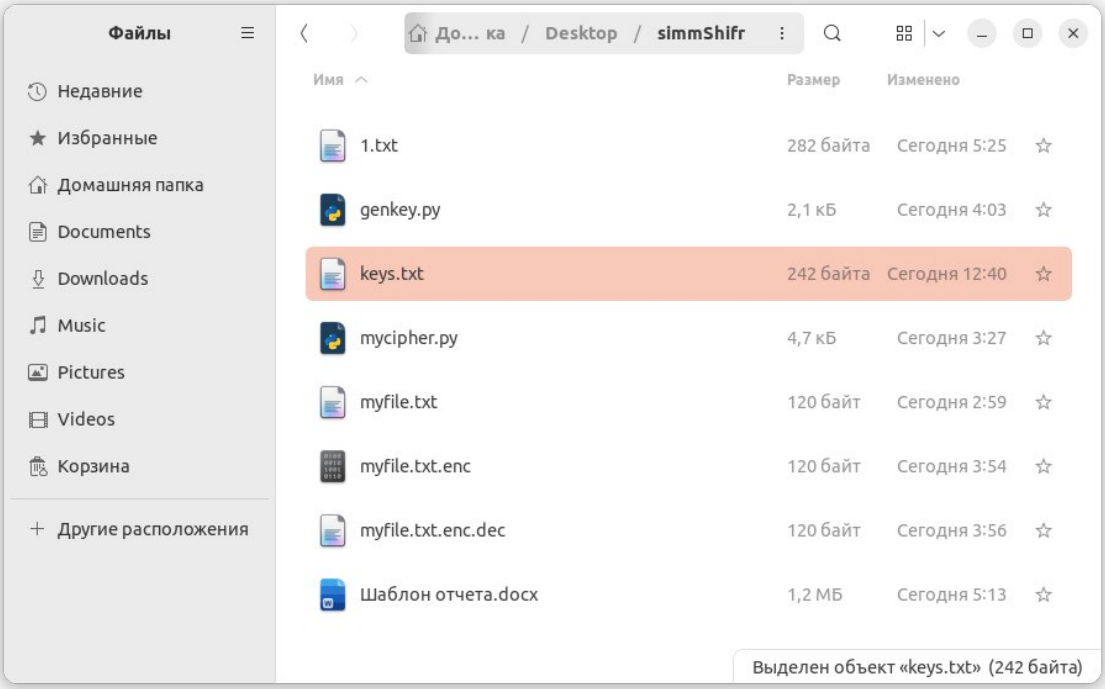

Рисунок 11 – в папке появился текстовый файл keys.txt

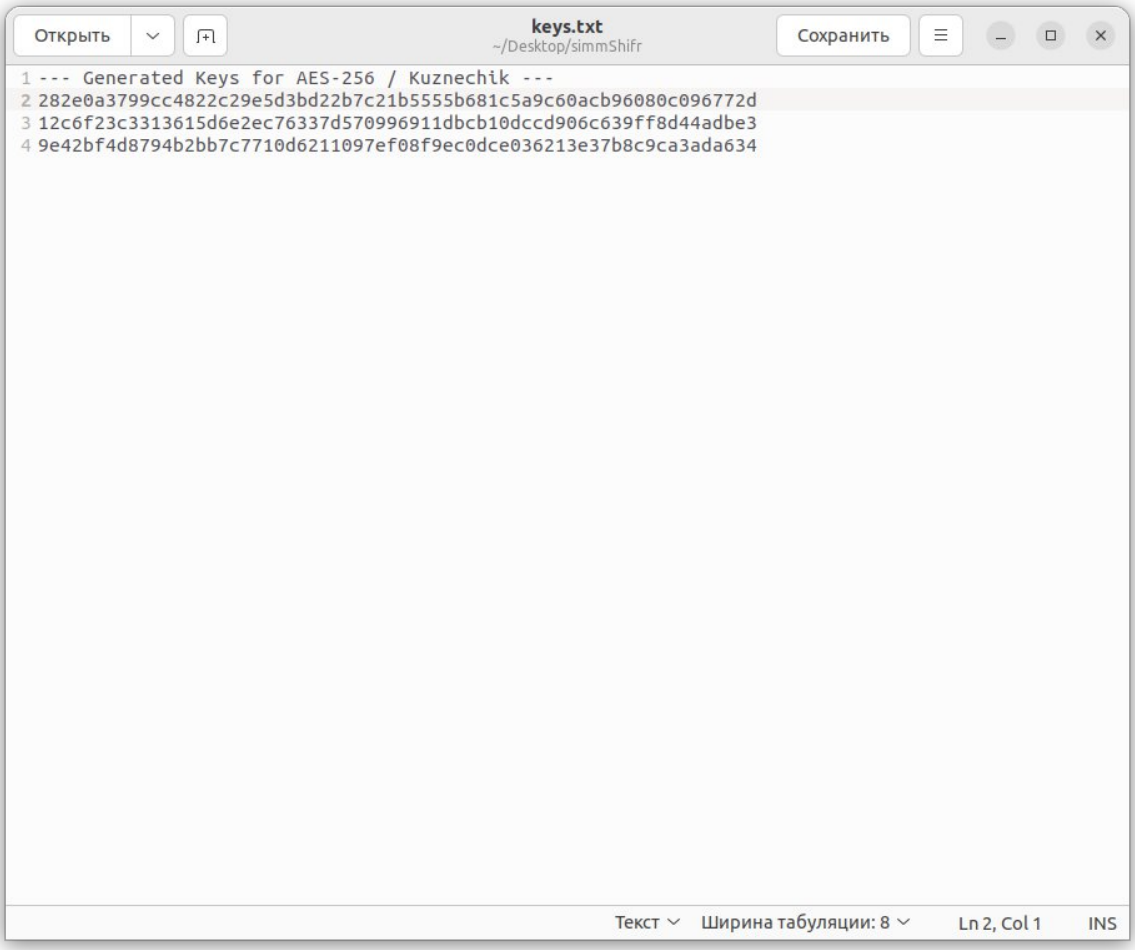

Рисунок 12 – содержимое файла keys.txt

5.8 Выводы о проделанной работе

В ходе выполнения работы было разработано вспомогательное программное средство — генератор ключей шифрования. Реализация выполнена на языке Python с использованием библиотеки secrets, что обеспечивает криптографическую стойкость генерируемых последовательностей (использование энтропии операционной системы). Программа автоматизирует процесс создания ключей длиной 128 и 256 бит, строго соблюдая формат шестнадцатеричной записи (HEX). Это позволило:

1) Исключить ошибки ввода недопустимых символов, возникающие при ручном наборе.
2) Гарантировать точное соответствие длины ключа требованиям алгоритмов AES и «Кузнечик».
3) Упростить процесс тестирования алгоритмов шифрования за счет сохранения наборов ключей в файл.

6 Список использованных источников:

1) ГОСТ Р 34.12-2015. Информационная технология. Криптографическая защита информации. Блочные шифры. – Введ. 2016-01-01. – Москва: Стандартинформ, 2015. – 25 с.

2) Бабаш А.В., Баранова Е.К. Анализ криптографической стойкости алгоритма Магма // Информационная безопасность. – 2018. – № 2. – С. 15-22.

3) Смирнов А.А., Петров И.В. Особенности программной реализации блочного шифра Магма // Сборник трудов конференции «RusCrypto'2017». – Москва, 2017. – С. 102-108.

4) Фомичёв, В. М. Криптографические методы защиты информации в 2 ч. Часть 1. Математические аспекты : учебник для вузов / В. М. Фомичёв, Д. А. Мельников ; под редакцией В. М. Фомичёва. — Москва : Издательство Юрайт, 2026. — 209 с. — (Высшее образование). — ISBN 978-5-9916-7088-3.

5) Фомичёв, В. М. Криптографические методы защиты информации в 2 ч. Часть 2. Системные и прикладные аспекты : учебник для вузов / В. М. Фомичёв, Д. А. Мельников ; под редакцией В. М. Фомичёва. — Москва : Издательство Юрайт, 2026. — 245 с. — (Высшее образование). — ISBN 978-5-9916-7090-6.

6) Омассон, Ж. -Ф. О криптографии всерьез / Ж. -Ф. Омассон ; перевод А. А. Слинкин. — Москва : ДМК Пресс, 2022. — 328 с. — ISBN 978-5-97060-975-0.

7) Свейгарт, Эл Криптография и взлом шифров на Python. : Пер. с англ. - СПб : ООО"Диалектика",2020. - 512 с. - Парал. тит. англ.

8) Лутц, Марк. Изучаем Python, том 1, 5-е изд.: Пер. с англ. — СПб.: ООО "Диалектика", 2019. —832 с. : ил. — Парад, тит. англ.

9) Лутц, Марк. Изучаем Python, том 2, 5-е изд. : Пер. с англ. — СПб. : ООО "Диалектика", 2020.— 720 с. : ил. — Парал. тит. англ.